

Final Project Proposal

Math Voyage: The Cargo Challenge

1st Edmond Kong
Net ID: eck67
eck67@cornell.edu

2nd Weiching Chen
Net ID: wc683
wc683@cornell.edu

3rd Leo Li
Net ID: zl2262
zl2262@cornell.edu

4th Tianze Lan
Net ID: tl929
tl929@cornell.edu

I. ASSUMPTIONS

- **Initial Position:** The TurtleBot (“cargo ship”) starts at the Shipping Port on a taped straight “river path” leading to the Receiving Port. Teams of two students stand at opposite ends. The flat classroom surface ensures stable movement, and facilitators can realign the robot if deviation occurs.

- **Sensor Data:** The robot relies on odometry for navigation and an external RFID module (via Raspberry Pi or ESP32) to detect tagged cargo pieces. LEDs (green, blue, red) and a start button provide feedback. A web interface displays robot status, cargo data, and team scores in real time.

- **Decision-Making:** RFID readings determine cargo load. If overloaded, the system triggers a red LED and “splash” sound; if valid, a blue LED and “arrival” sound play while updating scores.

- **Expected Behavior:** Students stay at their ports and observe robot cues. The robot moves autonomously after the start button is pressed. Facilitators may reset its position if needed to maintain fair play.

II. GAME MECHANICS

A. Setting

The game takes place in a classroom turned into a “harbor map” with two taped zones — Shipping and Receiving Ports — separated by a blue “river.” The TurtleBot acts as an autonomous cargo ship delivering math tiles between ports. Students work in teams, one at each port, fostering collaboration and communication while making math concepts visible and playful through robot movement, lights, and sounds.

B. Components

TurtleBot (ship), cargo tiles (numbers/operators/blanks), RFID module, LEDs, start button, timer, and web interface displaying cargo and scores.

C. Core Actions

- **Players:** Shipping Player loads number/operator tiles; Receiving Player unloads and arranges them into valid equations. Each correct equation earns 1 point.

- **Robot:** Moves autonomously between ports, verifies load

via RFID, and signals status with LEDs and sound cues.

D. Player Challenges

- **Cargo-limit guessing:** The sender must estimate how many tiles can safely fit before exceeding the ship’s load capacity.
- **Blank cargo blocks:** These blocks cannot be used for valid arithmetic operations, but they do contribute to the ship’s load capacity. Blank cargo blocks can be used by teams strategically to increase the sinking chances of the ship, or confuse the opposing team.
- **Turn-based cargo loading:** Senders in the Shipping Port alternate being the first person to load the ship, so teams can plan their actions strategically depending on their loading priority.
- **Hidden information:** The receiver has no knowledge of what was sent, requiring strategy and communication. Senders also have no knowledge of the types of blocks that are loaded by the opposing team.

E. Game Objectives

Teams earn points by forming valid equations. The game ends when cargo runs out or all possible equations are built; the highest-scoring team wins.

F. Game Rules

Players can:

- Load any combination of number, operator, and blank tiles each turn (within time limits).
- Communicate only through speech- they cannot show or pass cargo.
- Build and announce valid equations to gain points.

Players cannot:

- Exceed the declared cargo limit (overload causes the ship to “sink”).
- Touch or move the robot during its journey.
- Load the robot with objects other than cargo pieces.

Robot can:

- Move autonomously along the taped path.
- Detect load via RFID module and display visual feedback through LEDs.
- Trigger sound effects on arrival or overload.

Robot cannot:

- Choose cargo pieces or solve math problems . it only acts as a transporter and feedback provider.

G. Role of the Robot

The TurtleBot acts as the Cargo Ship, the centerpiece of the harbor world. it navigates autonomously between ports, carrying math cargo across the "river". Its LEDs and sounds bring personality to the experience:

- **Green LED:** ready to sail
- **Yellow LED:** loading phase of the ship
- **Blue LED:** successful delivery
- **Red LED + "splash" sound:** overloaded and sunk

The robot embodies a social character, a hard-working ship that students cheer for creating emotional engagement and making math feel like teamwork and adventure. The robot can interact with 2 or more participants simultaneously.

H. Interaction Between Students and Robot

Students load/unload cargo, press start, and observe LED and sound feedback. The robot and web interface offer real-time updates, encouraging teamwork and active learning.

I. Learning Objectives

Students practice arithmetic and problem-solving while developing teamwork and understanding feedback in autonomous systems — linking math and robotics to real-world logistics.

III. FINITE STATE MACHINE

A. State Machine and Robot Control Architecture

The control architecture for Math Voyage: The Cargo Challenge is implemented as a Finite State Machine (FSM) within a ROS-based multi-node system. The FSM runs in the fsm_node, which communicates with the following nodes through ROS topics:

- **motion_node:** Controls linear velocity commands (/cmd_vel) and odometry feedback (/odom).
- **rfid_node:** Reads cargo tag data through an external RFID module (connected to Raspberry Pi or ESP32) and publishes the detected cargo count.
- **io_node:** Handles hardware I/O: physical start/loading/reset buttons, and LED/sound output.
- **ui_node:** Hosts a web-based interface displaying the robot's current state, cargo data, and team score.

B. Use of Sensor Data for State Transitions

The FSM relies on multiple sensor inputs to determine transitions:

- **RFID Module:** Continuously detects cargo tags. During the CHECKING state, the tag count is compared to MAX_LOAD to determine if the transition should go to CRASH (overload) or MOVING (safe).

TABLE I
FINITE STATE MACHINE TRANSITIONS FOR THE CARGO CHALLENGE

State	Description	Transition Conditions	Next State
WAITING	Default idle state when the robot is not in active gameplay. The robot displays a green LED and waits for input.	"Load" button pressed → LOADING; "Start" button pressed → CHECKING; "Reset" button pressed → GOBACK.	LOADING / CHECKING / GOBACK
LOADING	Teams sequentially load cargo onto the robot. RFID continuously scans tags. Pressing the same "Load" button again indicates loading completion.	Second press of the "Load" button.	WAITING
CHECKING	The system reads RFID data to determine total cargo count.	$\text{Cargo} \leq \text{MAX_LOAD} \rightarrow \text{MOVING}$; $\text{Cargo} > \text{MAX_LOAD} \rightarrow \text{CRASH}$.	MOVING / CRASH
CRASH	Triggers explosion sound and red flashing LEDs, then turns back to the starting point.	Return complete.	WAITING
MOVING	The robot sails across the "river" for six seconds and reaches the opposite port.	Movement timer completed.	WAITING
GOBACK	Returns from opposite port when reset is triggered.	Return complete.	WAITING

- **Physical Buttons:** Button events are debounced (BTN_DEBOUNCE_MS) and trigger transitions between WAITING → LOADING or WAITING → CHECKING.
- **Odometry:** Used for timing and distance estimation during MOVING, CRASH, and GOBACK states to ensure consistent travel distance or duration.
- **No proximity or ultrasonic sensors** are used; all navigation is based on time and odometry since the classroom floor provides a controlled straight-line environment.

C. Transition Parameters and ROS Launch Setup

TABLE II
SYSTEM PARAMETERS FOR THE CARGO CHALLENGE

Parameter	Description	Example Value
MAX_LOAD	Maximum allowed number of cargo pieces before overload.	5
FORWARD_TIME_SEC	Full travel time across the river.	6 s
HALF_TIME_SEC	Half travel time for CRASH state.	3 s
RFID_STABLE_WINDOW_SEC	Time window for stable RFID readings before validation.	1.0 s
BTN_DEBOUNCE_MS	Hardware button debounce duration.	50 ms
LED_BLINK_PERIOD_MS	LED blink cycle during active states.	500 ms

A launch file (math_voyage.launch) can start all nodes simultaneously:

```
<launch>
  <node pkg="math_voyage" type="fsm_node.py"
    ↪ name="fsm_node" />
  <node pkg="math_voyage" type="motion_node.py"
    ↪ name="motion_node" />
  <node pkg="math_voyage" type="rfid_node.py"
    ↪ name="rfid_node" />
  <node pkg="math_voyage" type="io_node.py"
    ↪ name="io_node" />
  <node pkg="math_voyage" type="ui_node.py"
    ↪ name="ui_node" />
</launch>
```

TABLE III
FSM STATE TRANSITIONS: INPUT, OUTPUT, AND NEXT STATE

Current State	Input / Condition	Output / Action	Next State
WAITING	Load button pressed	LED: Green → Yellow; Web shows "Loading..."	LOADING
LOADING	Load button pressed again	Stop RFID scanning; update cargo list	WAITING
WAITING	Start button pressed	Begin RFID check; LED green flashing	CHECKING
CHECKING	Cargo > MAX_LOAD	Play "splash" sound; red LED flash	CRASH
CHECKING	Cargo ≤ MAX_LOAD	Play "departure" sound; LED blue	MOVING
CRASH	After 3 sec forward + 3 sec return	Stop motor; LED green	WAITING
MOVING	After 6 sec forward	Play "arrival" sound; LED blue	WAITING
WAITING	Reset button pressed	Move backward for 6 sec	GOBACK
GOBACK	After 6 sec	Stop motor; LED green	WAITING

D. State Transition Table
E. FSM Figure

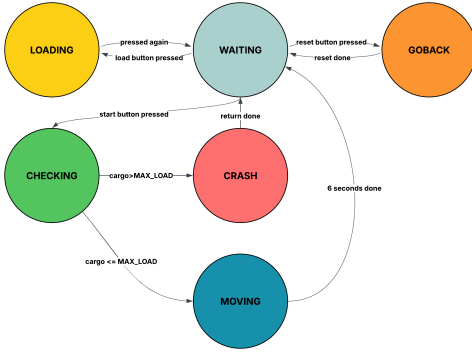


Fig. 1. Finite State Machine Figure

IV. ROBOT CONTROL ARCHITECTURE

A. Control Architecture Type and Justification

The robot control system for Math Voyage: The Cargo Challenge adopts a hybrid architecture, combining reactive behaviors for real-time responses (e.g., LED/sound feedback, button input) with deliberative planning for task sequencing (e.g., FSM-based navigation, load checking, and state control).

This hybrid structure is appropriate because the game requires both:

- **Immediate responses** to human interactions (button presses, cargo overload events).
- **Structured, sequential task flow** that represents a meaningful educational activity (loading → checking → moving → returning).

B. ROS2 Implementation and Node Communication

The control system is implemented in ROS 2 Humble on a TurtleBot4.

The architecture consists of multiple nodes communicating through ROS2 topics and services as shown below:

TABLE IV
ROS NODES AND COMMUNICATION TOPICS

Node	Function	Published Topics	Subscribed Topics
fsm_node	Central state machine managing robot behavior and transitions	/fsm/state, /led/cmd, /sound/cmd, /motion/cmd	/rfid/data, /button/event, /odom
motion_node	Controls the robot's velocity and handles motion timing	/cmd_vel	/motion/cmd
rfid_node	Reads RFID-tagged cargo via serial/USB interface	/rfid/data	—
io_node	Handles button inputs and LED/sound outputs	/button/event	/led/cmd, /sound/cmd
ui_node	Hosts the web-based dashboard for game monitoring	/ui/status	/fsm/state, /rfid/data

Example Message Flow:

- A student presses the Start button → io_node publishes /button/event.
- fsm_node receives it, verifies current state (WAITING → CHECKING), and requests an RFID scan.
- rfid_node publishes cargo data → fsm_node evaluates overload → publishes to /led/cmd and /sound/cmd.
- If the load is valid, fsm_node publishes /motion/cmd to motion_node, which translates it into /cmd_vel commands for TurtleBot4's base.
- ui_node subscribes to /fsm/state and /rfid/data to visualize the process on the classroom display.

C. Parallel and Sequential Components

The control logic combines sequential task planning with parallel feedback loops:

Sequential Components (FSM-based):

WAITING → LOADING → CHECKING → MOVING → WAITING

Each transition depends on button input, RFID readings, and timing conditions. These states execute sequentially to maintain structured gameplay.

Parallel Components (Reactive Behaviors):

- **LED and Sound Feedback:** Operate asynchronously to visualize robot states (e.g., red = overload, blue = arrival).
- **Web UI Updates:** Runs continuously in parallel, reflecting RFID and FSM status via ROS2 subscribers.
- **Safety Reset:** The Reset button interrupts any current motion sequence at any time, ensuring safety in a classroom.

D. Control Architecture Figure

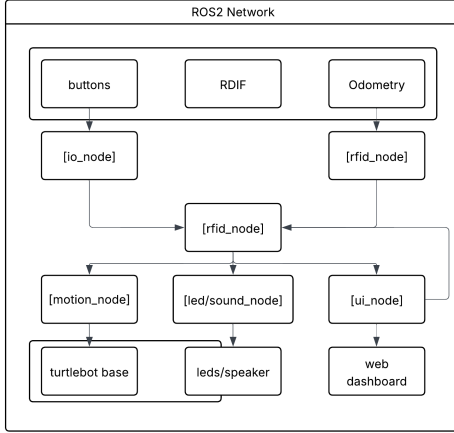


Fig. 2. Control Architecture Figure

V. ROBOT DESIGN AND PROTOTYPING

A. Robot Design and Form

The robot’s embodiment resembles a cargo ship navigating a miniature classroom harbor. The TurtleBot4 functions as a “ship” that delivers number and operator cargo blocks between two ports.

Early ideation involved sketching and cardboard prototyping to guide the placement of the cargo bay, LEDs, and sensors for visibility and interaction while keeping the design lightweight for smooth movement.

The final prototype is 3D-printed with a transparent upper deck, allowing students to see the cargo and internal lights. The deck integrates the RFID module, LED strips, and button interface, all visible and accessible to participants. Dynamic LEDs shift from blue to red to indicate sailing or sinking, while audio cues signal crashes or arrivals—bringing the ship to life as an engaging classroom character.

B. Fabrication and Materials

- **Material and Fabrication Choices:** The prototype is fully 3D-printed using PLA filament, combining multiple modular components designed for ease of assembly and interaction. The 3D-printed structure includes the ship’s hull, transparent upper deck, RFID mounts, LED brackets, and button holders. Hot glue and screws are used for assembly to ensure stability and durability.
- **Engagement Design:** Participants interact physically by loading cargo blocks and pressing buttons. The TurtleBot4 responds through LED color changes (blue = sailing, red = sinking) and sound effects, creating immediate feedback and reinforcing teamwork during gameplay.
- **Robot Body Use:** 3D-printed parts attach securely to the TurtleBot plate and anchor system, forming a modular “ship” body that enhances visibility and accessibility while maintaining smooth mobility.

C. Budget and Supplies

TABLE V
BUDGET AND SUPPLIES

Item	Purpose / Justification	Qty	Price
tRGB LED strip (WS2812B)	Visual feedback for sailing/sinking states	1	3 \$
PLA filament (1.75mm)	Print ship body and mounts	1	10 \$
Transparent PLA (50 g)	Visible deck for LEDs	1	7 \$

VI. USER STUDIES

A. Procedure

Facilitators welcome students, introduce the TurtleBot4 “cargo ship,” and explain the Math Harbor storyline and objectives. A brief demo shows how to load cargo blocks, press buttons, and form equations at the Receiving Port. Teams of two students play one full round while facilitators record timing and interactions. Afterward, participants complete a short survey on fun, difficulty, and engagement.

Team Roles:

- **Facilitator:** guides players and explains rules
- **Recorder:** logs data and timing
- **Tracker:** updates scores and monitors system

B. Game Testbed Setup

The classroom is taped into a mini harbor with Shipping and Receiving Ports, ocean sound effects, and labeled cargo areas. Pre-study setup includes calibrating odometry, testing buttons, RFID sensors, LEDs, and sound cues. FSM variables include `MAX_LOAD = 5`, `FORWARD_TIME = 6s`, and `RESET_TIME = 6s`.